ZoneSort: Cloud, Dashboard & ML

CS3237 Introduction to IoT · AY2026/27 Sem 1 · Group 3 · Check-in 1 progress report
Component owner: Chang Shu Kai · Status as of 19 Oct 2026

ZoneSort sorts parcels for HDB blocks. A courier loads parcels into a feeder, the gateway reads each QR label, and two ESP32 nodes route the parcel to **Zone A** (floors 1–5), **Zone B** (floors 6–10) or **Manual**. My part is everything after the sort: the shared event schema, storing every event, a live dashboard, and an ML model that **forecasts next-day parcel volume per block** so supervisors can plan how many couriers to send.

1. Progress at a glance
2. Where the cloud fits
3. Event schema and design decisions
4. Ingestion: MQTT → MySQL
5. Dashboard: Django API + React
6. API reference
7. ML: next-day forecasting
8. Results

1. Progress at a glance

Item	Status	Notes
Event schema (topics, payloads, tables)	Done	Agreed by the team
MySQL setup, accounts, remote read access	Done	Read-only `viewer` account over Tailscale
MQTT subscriber + fake-event generator	Done	Tested end to end through Mosquitto
Django REST API + React dashboard	Done	Live counts, faults, forecast panel
Traffic simulator, features, 3 models, evaluation	Done	Poisson regression selected
Integration with real Node 1 / Node 2 / gateway	Next	As soon as the firmware publishes; the schema is fixed
Collect real prototype runs; calibrate simulator	Before Check-in 2	Match the simulator's rates to measured throughput
Re-evaluate with real days included; live inference in demo	Before 13 Nov	Forecast is served live from the API

Code: `cloud/` (schema.sql, subscriber.py, fake_events.py, backend/, frontend/) and `ml/` (simulate.py, features.py, train.py, forecast.py) in the group repository.

2. Where the cloud fits

Sorting happens **on the ESP32s and never waits for the cloud**; if a zone lookup times out, the parcel goes to Manual. The cloud only listens, stores and analyses, so a cloud failure can never stop the belt. This follows the brief's split: real-time decisions at the edge, long-term analytics in the cloud.

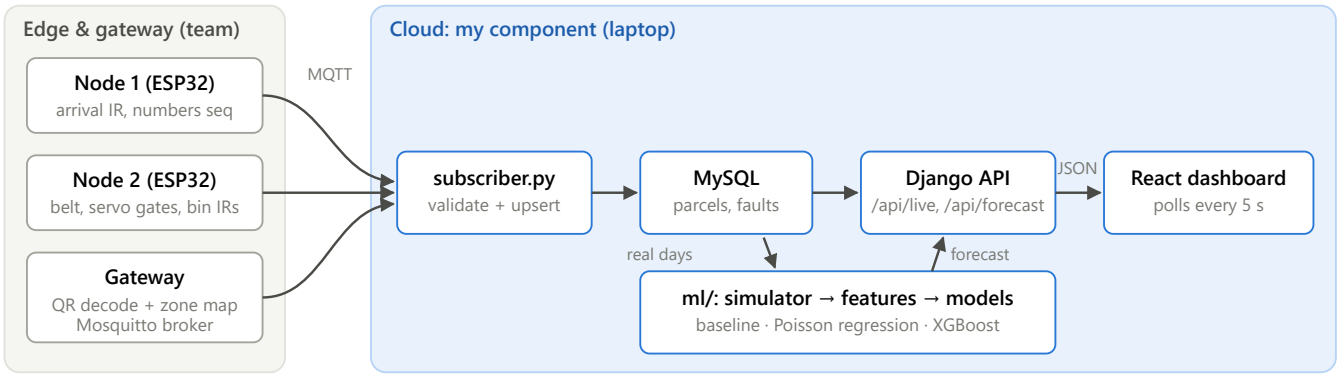


Figure 1. Data flow. Every node publishes to the gateway's MQTT broker; the subscriber stores events in MySQL; Django serves a REST API that the React dashboard polls. The ML pipeline trains on simulated history plus the real days stored in MySQL.

3. Event schema and design decisions

The schema is the contract between all five members: Node 1, Node 2 and the gateway publish in it, and the cloud stores it. I drafted it first so that firmware and gateway work could start against a fixed format; **the team has agreed to it**. Each rule below came from a constraint of the hardware.

Decision	Why
`seq` is the parcel key, not the label ID	Unreadable parcels have no `parcel_id`, but they still need a row. Node 1 numbers every arrival 1, 2, 3 … and every message about that parcel, including the ESP-NOW message from Node 1 to Node 2, carries `seq`.
Sessions are assigned by the cloud	`seq` restarts at 1 when Node 1 reboots. The subscriber starts a new session (`YYYYMMDD-HHMMSS`) each time Node 1 reports `online`, so a reboot can never overwrite earlier parcels. Key = `(session, seq)`.
Nodes send no timestamps	The ESP32s have no real-time clock. The subscriber stamps each event on arrival, which is accurate to a few ms on the LAN.
One row per parcel, built by upserts	Arrival, scan and sort events arrive separately and possibly out of order. Each one upserts only its own columns into the same row, so no joins are needed and the order doesn't matter.
`zone` and `routed` are separate	`zone` is what the gateway decided; `routed` is the gate Node 2 actually fired. A scan that arrives too late can't overwrite what physically happened, and a mismatch shows up as a timeout.
QoS 1 + retained Last-Will status	Parcel and fault events must not be lost (QoS 1). Each node's `status` topic is retained and set as its MQTT Last Will, so a node that drops off shows `offline` without sending anything.

MQTT topics

Topic	Publisher → subscribers	Payload
`zonesort/node1/arrival`	Node 1 → gateway, cloud	`{"seq": 17}`
`zonesort/gateway/flip`	gateway → Node 1	`{"seq": 17}`: label not found, flip and rescan
`zonesort/gateway/scan`	gateway → Node 1, cloud	`{"seq":17, "parcel_id":"P0042", "block":"123A", "floor":7, "unit":"07-123", "scan":"ok", "zone":"B", "flips":1}`
`zonesort/node2/sorted`	Node 2 → cloud	`{"seq":17, "routed":"B", "reason":null, "result":"confirmed"}`

`zonesort/fault`	any → cloud, Telegram	`{"source":"node2", "type":"jam", "seq":17, "detail":"no IR at gate B after 3 s"}`
`zonesort/<node>/status`	each node (retained, LWT)	`online` / `offline`

`scan` is `ok`, `unreadable` (no QR even after a flip) or `unknown` (QR read but block/floor not in the map); anything other than `ok` goes to `MANUAL`. `flips` counts flips before the final scan, so the share of parcels with `flips > 0` measures how often the flip mechanism saves a parcel from Manual.

Tables (MySQL)

`parcels`: one row per parcel, `UNIQUE(session, seq)`

`session`, `seq`	key (set by subscriber / Node 1)
`parcel_id`, `block`, `floor`, `unit`	from the label (NULL if unreadable)
`scan`, `flips`, `zone`	gateway's result and decision
`routed`, `reason`, `result`	what Node 2 did; `confirmed` / `missed` from bin IR
`arrived_at`, `scanned_at`, `sorted_at`	`DATETIME(3)`, stamped by subscriber

`faults`: one row per fault event

`session`	run it happened in
`source`	`node1` / `node2` / `gateway`
`type`	`jam` / `missed_ir` / `scan_fail_repeat` / …
`seq`, `detail`	which parcel, free text
`at`	defaults to now

Index `(block, arrived_at)` supports the per-block daily counts the forecaster needs. Teammates get a read-only `viewer` account over Tailscale.

4. Ingestion: MQTT → MySQL

`cloud/subscriber.py` (Python, paho-mqtt + SQLAlchemy) subscribes to `zonesort/#` at QoS 1 and turns each message into one upsert (`INSERT … ON DUPLICATE KEY UPDATE`) that touches only the columns that topic owns.

- **Robust to bad input:** malformed messages (bad JSON, missing or non-integer `seq`, a zone outside A/B/MANUAL) are logged and dropped instead of crashing the service.
- **Safe restarts:** if the subscriber restarts while Node 1 is still up, it resumes the latest session, but only if that session saw a parcel in the last 10 minutes. Otherwise Node 1 may have rebooted in the meantime, so it starts a new session rather than risk overwriting rows.
- **Testable without hardware:** `fake_events.py` publishes a realistic run in exactly the schema format, including unreadable labels, unknown floors, timeouts and missed parcels. The full path (fake run → Mosquitto → subscriber → MySQL → API → dashboard) has been run end to end; Figure 2 shows the result.

5. Dashboard: Django API + React

Why Django + React

Choice	Reason
Django (backend)	The whole cloud side is Python (subscriber, pandas, scikit-learn, XGBoost), so the API imports the forecasting code directly, with no second service to run. Its ORM maps the existing tables as **read-only, unmanaged models**: Django never creates or migrates them, so `schema.sql` stays the single source of truth. It also gives us a REST interface, one of the communication options in the brief.

React (frontend, Vite + Recharts)	A separate frontend lets the page update live without reloading, works on a phone over Tailscale, and is decoupled from the backend: the gateway or a future agent could use the same JSON API. Recharts gives interactive tooltips and stacked/grouped bars with little code.
Polling every 5 s instead of WebSockets	At prototype scale (a parcel every few seconds) a 5 s poll is fast enough, needs no extra infrastructure, and survives server restarts. WebSockets would be the next step for higher rates.
Replaced Streamlit	A first prototype was a single Streamlit script. It was quick to build, but it had no API other components could call and little layout control, so I moved to Django + React.

For the demo a single server runs everything: `npm run build` once, then `python manage.py runserver` serves both the API and the built React app. The **Simulated** switch in the header reads the simulated dataset instead of MySQL, so the dashboard can be developed and shown without hardware.

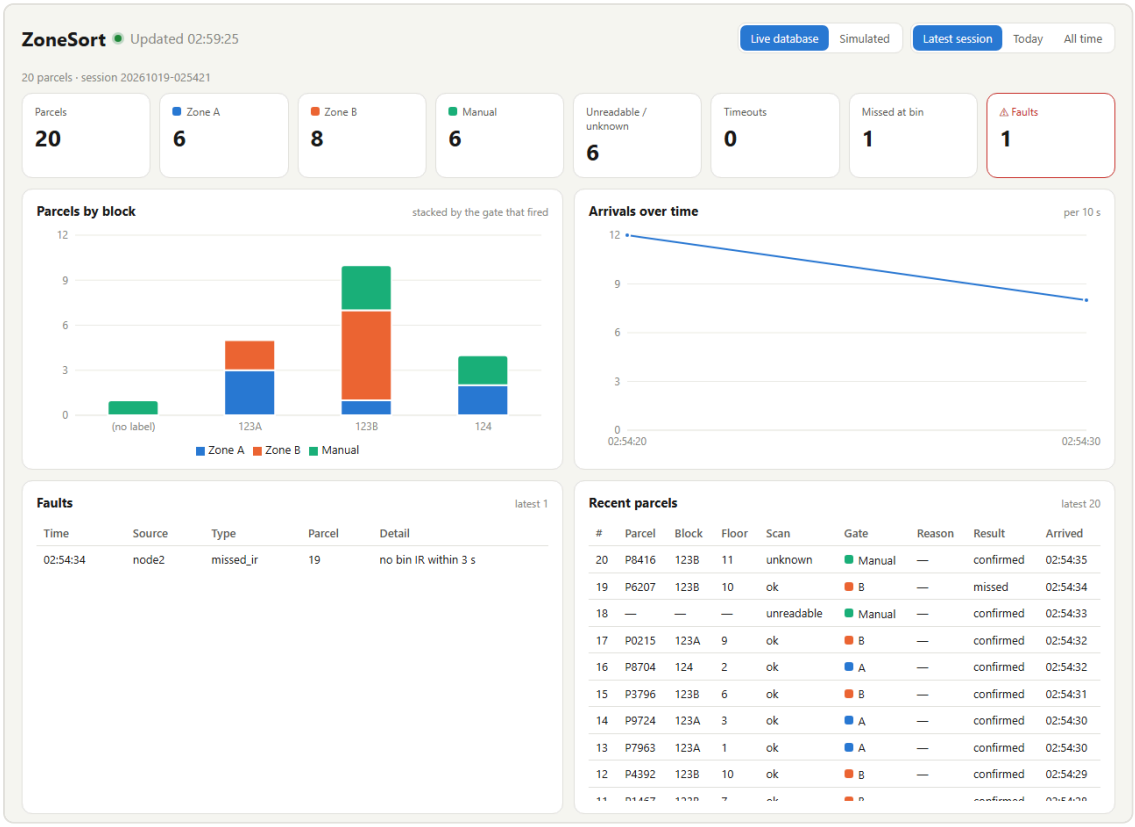


Figure 2a. Live section after a 20-parcel run from `fake_events.py` through the real broker, subscriber and MySQL: counts, parcels per block stacked by the gate that fired, arrivals, fault log and recent parcels.

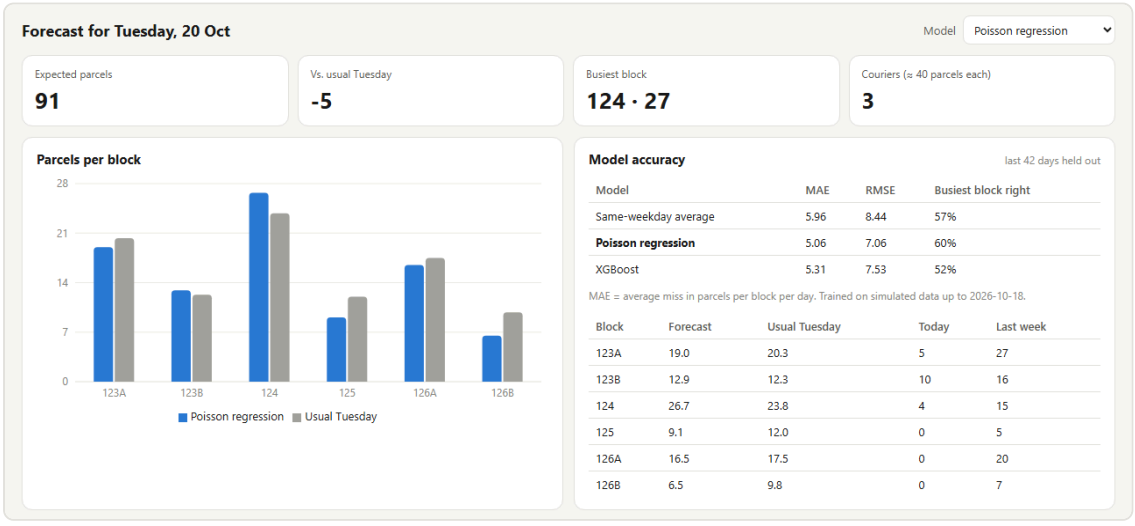


Figure 2b. Forecast section: tomorrow's parcels per block next to the usual count for that weekday, with each model's held-out accuracy.

6. API reference

Endpoint	Parameters	Returns
`GET /api/live`	`source` = `db` | `sim` `scope` = `latest` | `today` | `all`	KPI counts, parcels per block per routed zone, arrivals over time (bucket chosen from the time span: 10 s, minute, hour or day), the 50 latest parcels and faults.
`GET /api/forecast`	`source` = `db` | `sim` `model` = `poisson` | `xgb` | `baseline`	Tomorrow's date; per block the prediction, the usual count for that weekday, today's count and last week's; plus the saved test metrics. Cached for 60 s.

```
GET /api/live?source=db&scope=latest
{
  "session": "20261019-025421",
  "kpis": {"parcels": 20, "A": 6, "B": 8,
           "MANUAL": 6, "unreadable": 6,
           "timeouts": 0, "missed": 1, "faults": 1},
  "by_block": [{"block": "123A",
                "A": 3, "B": 2, "MANUAL": 0}, …],
  "arrivals": {"freq": "10 s", "points": […]},
  "recent": [{"seq": 20, "block": "123B",
              "floor": 11, "scan": "unknown",
              "routed": "MANUAL", …}, …],
  "faults": [{"type": "missed_ir", "seq": 19, …}]
}
```

```
GET /api/forecast?source=db&model=poisson
{
  "date": "2026-10-20",
  "model": "poisson",
  "blocks": [{"block": "124",
              "predicted": 26.7,
              "dow_avg": 23.8,   // usual Tuesday
              "lag1": 4,         // today
              "lag7": 15}, …],   // last Tuesday
  "metrics": {"source": "sim", "test_days": 42,
    "models": {"poisson": {"mae": 5.06,
               "rmse": 7.06, "busiest": 0.595}, …}}
}
```

Bad parameters return `400`. If MySQL is unreachable the API returns `503` with only the error *type*, never the driver's message, because a malformed connection string can echo part of the database password.

7. ML: next-day parcel forecasting

Problem

Predict, the evening before, **how many parcels each block will receive the next day**. Supervisors use it to decide how many couriers to send and which block to start with. The target is a **count** per (block, day), so this is a count-regression problem; regression is one of the methods covered in lectures.

Data: why a simulator, and how real data is used

A forecaster needs months of history to learn weekly patterns, but the prototype will only run on a handful of days before the demo. So `ml/simulate.py` generates a year of parcels **in exactly the `parcels` table format**, so the same code runs on simulated and real data. Each block's daily count is drawn as:

$$N \sim \text{Poisson}(\ \text{rate}_{\text{block}} \times \text{weekday} \times \text{trend} \times \text{sale} \times \text{noise}\ ), \quad \text{noise} \sim \text{Gamma}(12,\ 1/12)$$

Effect	Setting	Why
Block rate	6 blocks, 6–25 parcels/day	Blocks differ in size
Weekday	Mon ×1.35 … Sat ×0.80, Sun ×0.45	Monday catches the weekend backlog; few Sunday deliveries
Trend	+15% over the year	E-commerce growth
Sale days	9.9, 10.10, 11.11, 12.12: ×2.2, then ×1.6, ×1.3	Singapore's double-date sales; deliveries lag the sale
Noise	Gamma-Poisson mix (overdispersed)	Real daily counts vary more than a plain Poisson
Label outcomes	6% unreadable, 15% flipped, 3% timeout, 2% missed; floors 11–12 unknown	Exercise every path of the schema

> To meet the brief's requirement that ML is trained on our own sensor data, the default `both` mode uses **real MySQL events on every day the prototype ran and simulated traffic on all other days**. Real runs therefore feed both training and the live forecast. Simulated history only fills the gaps, which the brief allows as augmentation.

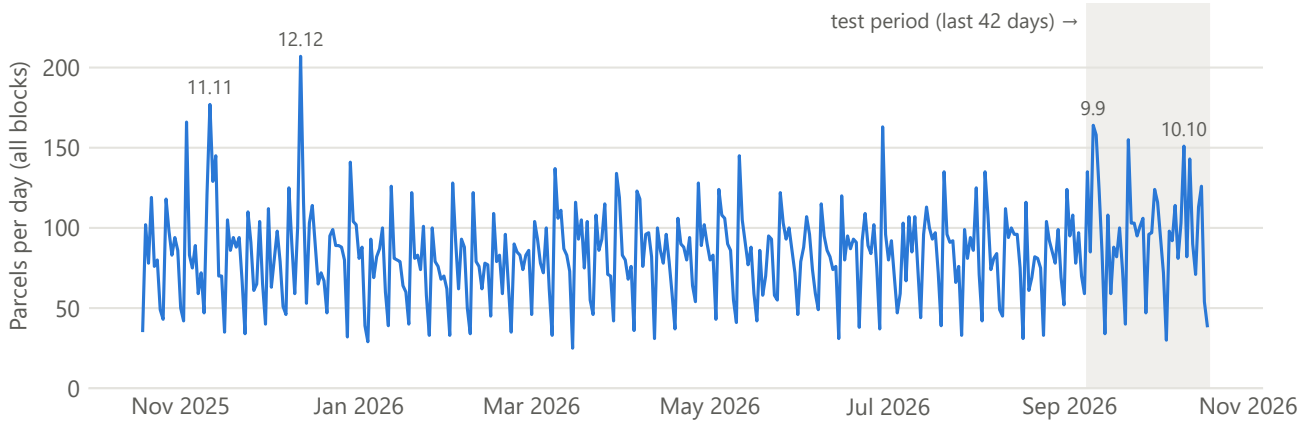


Figure 3. Simulated history: total parcels per day across all blocks (mean 84, range 25–207). The weekly cycle, growth and sale spikes are visible. The shaded area is the held-out test period.

Features

Every feature for day *d* uses only data up to day *d* − 1, so it is available when the forecast is issued and nothing leaks from the future.

Feature	Meaning
`block`, `dow`	Block ID and day of week (one-hot)
`sale_lag`	Days since a double-date sale (0, 1, 2, or none). A calendar fact, known in advance
`lag1`, `lag7`	Yesterday's count; same weekday last week
`roll7`, `roll28`	Mean of the previous 7 and 28 days (level of the series)
`dow_avg`	Mean of the same weekday over the previous 4 weeks (also the baseline's prediction)

Models and why each one

Model	Setup	Why it's here
Same-weekday average (baseline)	Predict `dow_avg`; no training	What a supervisor would do by hand. Any model has to beat it to be worth deploying.
Poisson regression	scikit-learn `PoissonRegressor` (log link, L2 α = 0.001); one-hot categories, log(1+x) standardised lags	Built for counts: predictions are never negative and variance grows with the mean. The log link makes effects **multiplicative** ("Mondays are 35% busier"), which matches how parcel volume behaves.
XGBoost	400 trees, depth 4, learning rate 0.03, subsampling 0.8, `count:poisson` objective	Can learn non-linear effects and interactions (e.g. sales hitting big blocks harder) without hand-made features. Tests whether extra flexibility helps.

Evaluation

Time-ordered split, never shuffled: train on the earlier 295 days, test on the **last 42 days** (7 Sep to 18 Oct 2026, 252 block-days), which include the 9.9 and 10.10 sales. The first 28 days are dropped because their 28-day features aren't defined yet. The saved models are then refit on all data for live use.

- **MAE**: average absolute miss in parcels per block per day. Easy to explain to a supervisor.

- **RMSE**: penalises large misses such as under-predicting a sale day.
- **Busiest-block accuracy**: share of test days where the model picks the block that really was busiest. This is the operational question: where to send the first courier.

8. Results

Model	MAE	RMSE	Busiest block correct
Same-weekday average	5.96	8.45	57%
Poisson regression	**5.06**	**7.06**	**60%**
XGBoost	5.31	7.53	52%

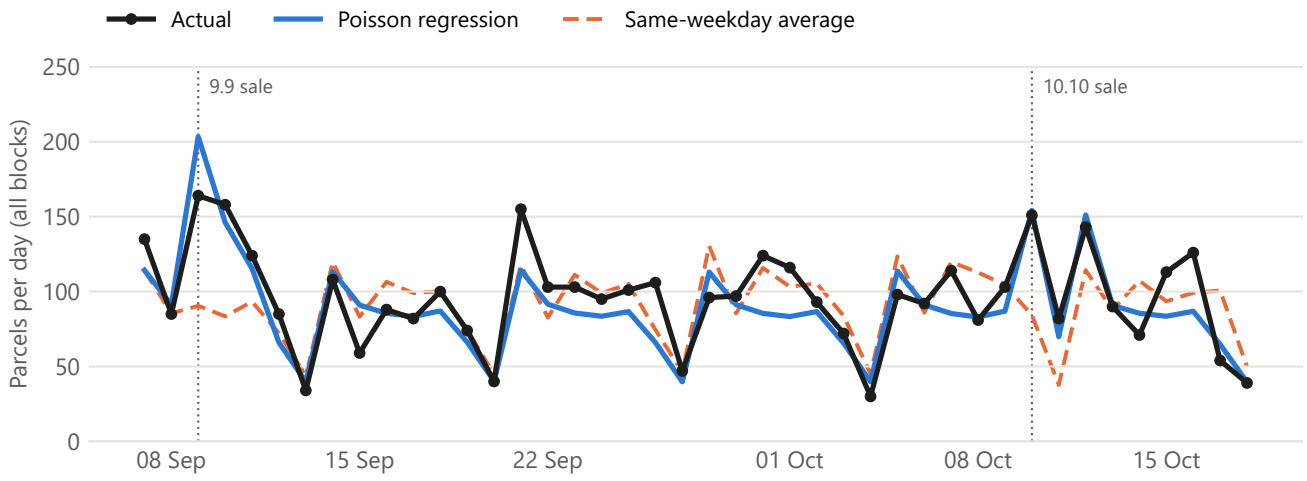


Figure 4. Test period, summed over all blocks. The baseline can't see sales coming (10.10: 151 actual, 84 predicted); Poisson regression anticipates them through the `sale_lag` feature (154 predicted), though it over-shoots 9.9 (203 vs 164).

- **Poisson regression is best on every metric**, cutting MAE by 15% over the baseline. It is the dashboard's default.
- **Most of the gain is on sale days.** On the six sale and post-sale days in the test period, the MAE is 6.0 for Poisson vs 9.6 for the baseline; on ordinary days it is 4.9 vs 5.4. Knowing the calendar matters more than any lag feature.
- **Why XGBoost doesn't win:** the data is multiplicative, which a log-link linear model captures exactly, and 295 training days × 6 blocks is small for boosted trees. A flexible model isn't automatically better; the baseline comparison makes that visible.
- **Busiest-block accuracy is capped near 60%** because two blocks (123A and 124) are often within day-to-day noise of each other.

> **Limitation.** These numbers come from simulated data, so they show the pipeline works and the models behave sensibly, not real-world accuracy. Simulated daily rates (about 85 parcels/day) are also far above what the prototype handles in a demo run (about 20), so the next step is to calibrate the simulator to the prototype's real throughput and re-evaluate on days that include real runs.